Code Structure — Intelligent University Course Finder

Project Root

```
Capstone_Project/
│
├── .env                        # Environment variables (OPENAI_API_KEY, QDRANT_PATH,
LLM_MODEL, etc.)
├── .gitignore                  # Git ignore rules
├── config.py                   # Centralised config — loads .env, CrossEncoder strings,
exposes constants
├── logger.py                   # Logger factory — get_logger(name) → per-module rotating file
+ console handler
├── requirements.txt            # All Python dependencies
│
├── data/                       # Raw source data
│   └── coursera_courses.csv    # Original Coursera dataset (title, skills, difficulty,
duration, org, rating…)
│
├── ingestion/                  # One-time data ingestion pipeline
│   └── ingest.py               # Loads CSV → builds parent + child + BM25 docs → upserts to
Qdrant → pickles BM25
│
├── storage/                    # Persisted index files (auto-created by ingest.py)
│   ├── qdrant_db/              # Qdrant local vector database
│   │   ├── courses_parents     # 402 points — full composite course text
│   │   └── courses_children    # 1206 points — identity / skills / description field chunks
│   └── bm25_index.pkl          # Pickled BM25 index for keyword retrieval
│
├── retrieval/                  # Retrieval layer
│   └── hybrid_retriever.py     # hybrid_search() — Qdrant + BM25 → RRF fusion → parent fetch
→ filters → cross-encoder rerank
│
├── guardrails/                 # Input safety layer
│   ├── __init__.py
│   └── validator.py            # GuardrailValidator — Presidio PII redaction + LangChain
intent validation
│
├── app_agents/                 # AI agent pipeline (OpenAI Agents SDK)
│   ├── __init__.py             # Bootstraps SDK after load_dotenv(); re-exports Agent,
Runner, function_tool
│   ├── tools.py                # All @function_tool definitions used by agents:
│   │                           #   extract_intent, search_courses, report_skill_gap,
│   │                           #   report_career_alignment, report_recommendation
│   ├── intent_agent.py         # IntentAgent — parse(query) → extracts intent + retrieves top
10 courses
│   ├── skill_gap_agent.py      # SkillGapAgent — analyse(level, courses) → missing
prerequisite skills
│   ├── career_agent.py         # CareerAgent — align(goal, courses) → career track + top
relevant courses
```

```
│   ├── sequencer.py            # CourseSequencer — sequence(courses) → difficulty-sorted
LearningPath (no LLM)
│   └── advisor_agent.py        # LearningAdvisorAgent — advise(...) → summary + ordered path
+ study tips
│
├── api_gateway/                # API Gateway — single frontend entry point (port 8080)
│   ├── __init__.py
│   └── gateway.py              # FastAPI proxy — forwards /recommend to backend microservice
on port 8000
│                               # Adds request logging, timing headers, graceful 503/504 error
handling
│
├── api/                        # Backend microservice (port 8000)
│   ├── __init__.py
│   ├── main.py                 # FastAPI app entry point — registers router, CORS, /health
endpoint
│   ├── schemas.py              # Pydantic I/O models: RecommendRequest, RecommendResponse,
CourseSchema, etc.
│   ├── core/
│   │   └── recommend_logic.py  # Pipeline orchestrator — runs agents sequentially/in parallel
with asyncio.gather
│   ├── routes/
│   │   └── recommend_route.py  # APIRouter — POST /recommend endpoint definition
│   └── utils/
│       └── helpers.py          # to_course_schema() — converts raw dict to CourseSchema
Pydantic model
│
├── frontend/
│   └── careerguidefrontend/    # React 19 + Vite frontend (npm run dev → localhost:5173)
│       ├── index.html          # HTML entry point
│       ├── vite.config.js      # Vite build config
│       ├── package.json        # npm dependencies (react, react-dom, lucide-react)
│       └── src/
│           ├── main.jsx        # React DOM root entry point
│           ├── index.css       # Global design system — CSS variables, minimalist light theme
│           ├── App.jsx         # Root component — manages state, calls API Gateway, renders
results
│           ├── App.css         # App-level layout styles — hero, animation, results container
│           └── components/
│               ├── SearchBar.jsx    # Search input field + loading spinner button
│               ├── SearchBar.css
│               ├── CareerCard.jsx   # Displays career track badge and alignment reasoning
│               ├── CareerCard.css
│               ├── AdvisorCard.jsx  # Displays advisor summary, study tips, and skill gap
tags
│               ├── AdvisorCard.css
│               ├── CourseList.jsx   # Difficulty-staged course pathway with card tiles +
links
│               └── CourseList.css
│
├── test/                       # Test scripts
│   ├── check_qdrant.py         # Utility — inspects Qdrant collections and sample points
```

```
│   ├── test_guardrails.py      # Tests GuardrailValidator — PII redaction + intent rejection
│   ├── test_intent_agent.py    # Tests IntentAgent end-to-end with 4 sample queries
│   └── test_retriver.py        # Tests hybrid_search with 6 sample queries and filter
combinations
│
├── Artifacts/                  # Project documentation and diagrams
│   ├── Architecture Diagram.png
│   ├── Data Ingestion.png
│   ├── Flow Diagram.png
│   ├── design_decisions.md         # All major technical design decisions with rationale
│   ├── design_decisions.pdf
│   ├── CODE_STRUCTURE.md           # This file
│   ├── CODE_STRUCTURE.pdf
│   └── University Course Finder – Intelligent Course Discovery System.pdf
│
└── logs/                       # Per-module log files (auto-created at runtime)
    ├── intent_agent.log
    ├── skill_gap_agent.log
    ├── career_agent.log
    ├── advisor_agent.log
    ├── agent_tools.log
    ├── guardrails.log
    ├── recommend_logic.log
    └── api_gateway.log
```

Key Entry Points

Command	Purpose
`python ingestion/ingest.py`	One-time data ingestion — builds Qdrant + BM25 index
`uvicorn api.main:app --port 8000 --reload`	Start backend microservice
`uvicorn api_gateway.gateway:app --port 8080 --reload`	Start API Gateway
`npm run dev` *(in `frontend/careerguidefrontend/`)*	Start React frontend
`python test/test_intent_agent.py`	Test IntentAgent in isolation
`python test/test_guardrails.py`	Test PII redaction + intent validation
`python test/test_retriver.py`	Test hybrid retriever with sample queries

Request Flow (file-by-file)

```
src/App.jsx
  └─ POST http://localhost:8080/recommend
      └─ api_gateway/gateway.py
          └─ POST http://localhost:8000/recommend
```

```
└─ api/routes/recommend_route.py
    └─ api/core/recommend_logic.py
        ├─ guardrails/validator.py        (PII + intent check)
        ├─ app_agents/intent_agent.py
        │      └─ app_agents/tools.py
        │            └─ retrieval/hybrid_retriever.py
        ├─ app_agents/skill_gap_agent.py  ─┐ parallel via
        ├─ app_agents/career_agent.py     ─┘ asyncio.gather
        ├─ app_agents/sequencer.py        (pure Python, no LLM)
        └─ app_agents/advisor_agent.py
              └─ api/schemas.py → RecommendResponse
                  └─ api_gateway/gateway.py → App.jsx
```